MERN Backend Assessment

API Consumption, JWT Authentication, and Data Handling

1. Objective

This assessment evaluates your understanding of:

- REST API consumption using Axios
- Async/Await and Promise handling
- JWT-based authentication
- Request headers and body handling
- Data caching in memory
- Express routing
- Clean backend architecture
- Deployment to Render

2. Assessment Overview

You will be provided with two external APIs:

1. **Public API** – Issues a JWT token when provided with your student ID
2. **Private API** – Returns dataset only when a valid JWT token is sent in the Authorization header

Your task is to:

1. Request a token using your student ID
2. Use the token to access the private API
3. Fetch the dataset once at server startup
4. Cache the dataset in memory
5. Build five custom endpoints using the cached data
6. Deploy your server on Render
7. Submit both Render and GitHub URLs

3. Step 1 – Obtain JWT Token (Public API)

Endpoint (Provided)

POST /public/token

Request Body

```
{
 "studentId": "YOUR_REGISTER_NUMBER",
 "set": "setA",
 "password": "YOUR_PASSWORD"
}
```

Response Format

```
{
 "token": "JWT_TOKEN_STRING"
}
```

Notes:

- The request must be sent using POST.
- The studentId, set and password must be sent in the request body (password given on exam day).
- Token validity is limited to 5 minutes.
- Invalid studentId will result in 403 Forbidden.
- The set must match your question paper (setA – setF).

4. Step 2 – Access Private API

Endpoint (Provided)

GET /private/data

Required Header

Authorization: Bearer <JWT_TOKEN>

Response behavior:

- Valid token → Dataset returned
- Invalid or expired token → 401 Unauthorized

5. Implementation Rules

You must follow the rules below strictly:

1. Fetch the dataset once at server startup.
2. Store the dataset in memory.
3. Use the cached dataset for all custom endpoints.

Not allowed:

- Calling the private API inside each route.
- Hardcoding the token.
- Bypassing the public API.
- Sharing tokens between students.

Failure to follow these rules will result in mark deduction.

6. Endpoints to Implement

Using the cached dataset, implement the five endpoints given in your set's question paper.

Each set (A–F) has its own dataset and base route:

Set	Dataset	Base route
A	Hospital Patients	/patients
B	E-Commerce	/products
C	HR	/employees
D	Orders	/orders
E	Course Enrollment	/enrollments
F	Academic	/students

What every set's endpoints test

- Highest / top record – found with reduce or sorting; derived totals calculated dynamically.
- Lookup by route parameter (e.g. /:id) – validate the input; return 404 when not found.
- Filter by query parameter – return only matching records; 400 when the parameter is missing.
- Aggregation / summary – counts, totals or averages using reduce, rounded as specified.
- Set-specific logic – e.g. longest stay, discounted price, most enrolled course.

Rules for all endpoints

- Use only the cached dataset and never modify the original data.
- Return proper status codes with JSON error messages, e.g. { "message": "..." }.
- Exact routes, response formats and examples are given in your question paper.
- Each endpoint carries 5 marks (see Section 9).

7. Technical Requirements

Your project must include:

- Express
- Axios
- Async/Await
- Proper error handling
- Express.json() middleware
- Clean folder structure (recommended: routes, controllers, services)

Recommended architecture:

routes/

controllers/

services/

app.js

server.js

8. Deployment Requirements

You must:

1. Deploy your project on Render.
2. Ensure application uses process.env.PORT.
3. Ensure the server starts without manual intervention.
4. Submit:
    - GitHub repository URL
    - Render live URL

9. Evaluation Criteria (100 Marks)

Criteria	What is assessed	Marks
Public API – token retrieval	POST /public/token with studentId in body; token taken from response, not hardcoded	15
JWT handling & header usage	Authorization: Bearer <token> sent correctly; 401 / expired token handled	15
Private API integration & caching	Dataset fetched once at startup, kept in memory, reused by every endpoint	20
Endpoints (5 × 5 marks)	Each set-specific endpoint gives correct output; route/query params as specified	25
Code structure & deployment	Clean modules, async/await, express.json(), process.env.PORT, live on Render	15
Error handling & status codes	400 invalid input, 404 not found, 503 before data loads; server never crashes	10
Total		**100**

10. Academic Integrity

- Each student receives a unique token tied to their student ID.
- Tokens expire after 5 minutes.
- Sharing tokens is considered malpractice.
- Any attempt to bypass authentication will result in disqualification.

11. Submission Instructions

Submit the following before the deadline:

1. GitHub repository link
2. Render deployment link

Late submissions will not be accepted.